

ReliaGuard Studio

Project Repository Summary

Ahan Bhatt

June 18, 2026

Purpose

ReliaGuard Studio is a production-style AI evaluation platform for detecting overreliance, underreliance, and unsafe human-AI decision behavior. The central product premise is simple: AI assistance can improve average accuracy while still causing harmful reliance failures. A user may accept incorrect AI advice, reject correct advice, become overconfident after flawed support, or need a verification prompt before finalizing a decision.

The platform turns those hidden failure modes into deployable product signals: reliance-state probabilities, calibrated harmful-reliance risk, symbolic rule traces, counterfactual explanations, conformal thresholds, and selective-gating policy outputs. It is not a clinical, diagnostic, cognitive-decline, or causal intervention system. It is an AI evaluation and model-risk analysis platform grounded in public human-AI interaction evidence.

The underlying analysis integrates 43,263 public records from 2,229 participants or students across five datasets: HAIID, CHI 2023 DKE, ConvXAI, Pardos/Bhandari ChatGPT tutoring, and FLoRA IPS. The public repository is organized as a product-oriented code release. Manuscript drafts, research-paper figures, raw datasets, generated results, submission audits, and reviewer materials are intentionally ignored by Git. This protects unpublished paper content and avoids accidental redistribution of third-party data.

High-Level Architecture

The repository now uses a full-stack product architecture. The Next.js frontend in `apps/web` provides the landing page, interactive case simulator, conformal-gating dashboard, model-evaluation lab, policy-simulation page, data/reproducibility page, and safety/limitations panel. The FastAPI backend in `apps/api` exposes reliance prediction, explanation, conformal threshold, policy simulation, ablation, dataset, and model-card endpoints. The Python package under `src/reliaguard_studio` remains the source of truth for data adapters, reliance-state logic, symbolic rules, evaluation, statistics, and model artifacts. A product-facing wrapper in `packages/relia_core` gives downstream services a smaller import surface.

Area	Role
<code>apps/web/</code>	Next.js, TypeScript, Tailwind product frontend for the public ReliaGuard Studio experience.
<code>apps/api/</code>	FastAPI product service endpoint that serves calibrated risk, rule traces, thresholds, policy outputs, and model cards.
<code>packages/relia_core/</code>	Product-facing wrapper around the Python reliance engine for clean imports and future packaging.

<code>config/</code>	Loads schema-driven YAML configuration and Pydantic schemas for the public reliance-analysis package.
<code>data/</code>	Defines canonical data schemas, synthetic benchmark utilities, storage helpers, and real public-dataset adapters.
<code>models/</code>	Implements baselines, fusion models, reliance-state models, and ReliaGuard-NS gating components.
<code>rules/</code>	Implements symbolic and fuzzy rules for overreliance, underreliance, confidence inflation, process-trace risk, and counterfactual explanations.
<code>evaluation/</code>	Provides metrics, calibration, GEE statistics, cross-dataset tests, policy simulation, conformal risk control, sensitivity analyses, and claims-audit utilities.
<code>visualization/</code>	Generates publication-style figures, source-data tables, contact sheets, render-safety checks, and visual-text audits. These outputs are ignored when they belong to the private paper package.
<code>api/</code>	Exposes the FastAPI application used by both the product API wrapper and legacy local demo routes.
<code>infra/</code>	Docker Compose and deployment scaffolding for running the web and API services together.
<code>docs/</code>	Architecture, deployment, model-card, portfolio, and project-summary documentation.
<code>study_platform/</code>	Provides a prospective validation scaffold with task definitions, randomization, simulation, quality checks, export, and analysis.
<code>theory/</code>	Encodes reliance-state decomposition and utility-bound helpers used by the method layer.

Repository Layout

Path	Description
<code>apps/web/</code>	Product frontend: landing page, simulator, gating dashboard, evaluation lab, policy page, data page, and limitations page.
<code>apps/api/</code>	Product API endpoint wrapping the FastAPI application.
<code>packages/relia_core/</code>	Lightweight product package facade over the reliance-state scoring logic.
<code>infra/</code>	Docker Compose and deployment notes for local full-stack operation.
<code>src/</code>	Main installable package. This is the core of the public release.
<code>tests/</code>	Unit, integration, paper-optional smoke, model, metric, rule, study-platform, and real-data pipeline tests.
<code>experiments/</code>	Lightweight command wrappers for asset generation and AIR-Bench-style synthetic stress tests.
<code>docs/</code>	Public docs for architecture, deployment, portfolio/resume positioning, model cards, and this project summary.
<code>notebooks/</code>	Reserved for exploratory notebooks. Core logic should stay in <code>src/</code> .
<code>paper/</code>	Private manuscript source, figures, source data, supplementary files, and PDFs. Ignored by Git.
<code>artifacts/</code>	Downloaded raw data, prepared data, experiment outputs, runtime databases, and generated reports. Ignored by Git.

<code>.private/</code>	Local-only archive for legacy prototype files, original documents, submission audits, dataset cards, model cards, and reviewer materials. Ignored by Git.
<code>.private/original_docs/</code>	Legacy/original project documents if present locally. Ignored by Git.

Data Layer

The data layer maps heterogeneous datasets into common tables for interactions, participants, tasks, and metadata. The canonical schema supports:

- dataset name and source metadata;
- participant and task identifiers;
- initial human response and final human response;
- advice value, advice source, and advice correctness where available;
- initial correctness and final correctness;
- confidence before and after advice where derivable;
- movement toward or away from advice;
- appropriate reliance, overreliance, underreliance, correct AI reliance, and correct self-reliance labels where the dataset supports them;
- learning-gain and process-trace features for datasets that are not advice-taking datasets.

Raw public datasets are downloaded into `artifacts/real_data/raw/`; prepared canonical tables are written to `artifacts/real_data/prepared/`. Both are ignored so that GitHub publication does not accidentally redistribute participant data.

Modeling Layer

The modeling layer includes standard statistical and machine-learning baselines plus neuro-symbolic components:

- logistic regression and calibrated logistic baselines;
- random forest and gradient-boosting baselines;
- MLP baselines where appropriate;
- symbolic-only rule models;
- weighted and learned fusion models;
- reliance-state neuro-symbolic models;
- ReliaGuard-NS conformal selective-risk control;
- uncertainty-aware gating and conservative offline policy simulation.

The intended contribution is not a clinical score or a universal classifier. The system makes reliance states measurable, auditable, and usable for cautious intervention design.

Symbolic Reasoning

The symbolic layer converts behavioral patterns into interpretable rule activations. Example rule families include:

- wrong-advice overreliance;
- correct-advice underreliance;
- confidence inflation after advice;
- correct self-reliance under disagreement;
- low-confidence beneficial reliance;
- process-trace warning and protective rules for GenAI writing workflows.

Rules produce explanation traces and counterfactual suggestions. For example, a case with high advice uptake, wrong advice, and increased confidence can be flagged as a confidence-inflated overreliance risk. The counterfactual explanation can identify which signal would need to change to reduce that risk.

Evaluation Layer

The evaluation layer provides reproducible analyses and quality gates:

- AUROC, AUPRC, F1, balanced accuracy, Brier score, and expected calibration error;
- regression metrics such as RMSE, MAE, and R^2 where learning or process outcomes are continuous;
- participant-aware, task-aware, random, and cross-dataset validation;
- GEE models with participant clustering for repeated-measures settings;
- bootstrap confidence intervals and split robustness;
- calibration curves, class-conditional calibration, and reliability summaries;
- rule ablations and model ablations;
- sensitivity to labels, thresholds, harm weights, and intervention burden;
- non-causal offline policy simulation and conformal selective-risk summaries.

The evaluation code explicitly distinguishes prediction, association, explanation, observational policy simulation, and causal intervention. Prospective causal claims require new approved human-subject validation.

API And Demo

The local FastAPI app serves the static research demo and exposes scoring/report endpoints. The primary static assets are bundled inside the package at `src/reliaguard_studio/frontend/static/`. The legacy root-level browser prototype was moved to private local storage because it is not needed for the public code release.

Prospective Validation Scaffold

The `study_platform/` module is a scaffold for future IRB-approved validation. It includes:

- task definitions;
- random assignment to assistance/gating conditions;
- simulated smoke-test data generation;
- data export and analysis;
- power-analysis helpers;
- recruitment and privacy validation gates;
- quality checks and readiness reports.

This scaffold does not imply that a real prospective study has been completed. It is infrastructure for future validation.

Important Commands

Command	Purpose
<code>python-mpipinstall-e.[dev]</code>	Install the package and test dependencies.
<code>nscasearch-datasets</code>	Display the dataset registry and integration decisions.
<code>nscadownload-real-data</code>	Download public datasets into ignored artifacts.
<code>nscaprepare-real-data</code>	Convert datasets into canonical tables.
<code>nscarun-real-experiments</code>	Run descriptive analyses, models, statistics, calibration, and policy outputs.
<code>nscarun-cross-dataset</code>	Refresh construct-transfer and cross-dataset analyses.
<code>nscarun-policy-evaluation</code>	Run conservative non-causal gating-policy simulations.
<code>nsca run-conformal-risk-control</code>	Run ReliaGuard-NS conformal selective-risk summaries.
<code>nscarun-sensitivity-analyses</code>	Refresh threshold, burden, label, calibration, and harm-weight sensitivity outputs.
<code>nscaserve-api</code>	Run the local research demo API.
<code>nscaserve-study</code>	Run the local prospective-study API scaffold.
<code>pytest</code>	Run the test suite.
<code>python-mcompileallsrc</code>	Check Python syntax compilation.

Publication Safety

The public repository is designed to avoid accidental publication of unpublished paper materials:

- `paper/` is ignored.
- `artifacts/` is ignored.
- `.private/` is ignored.
- root-level submission audits and paper-support files are ignored.

- raw downloaded datasets are ignored.
- generated figures, PDFs, source-data tables, and manuscript logs are ignored.

Before publishing, run:

```
git status --ignored
git check-ignore -v paper artifacts .private Docs
```

If these paths are ignored, GitHub publication should include the code package and public project summary but not the private manuscript package.

Known Boundaries

The code supports real public-data analyses, but it does not support every possible claim. The project does not make clinical or diagnostic claims. It does not claim delayed recall, transfer, or long-term learning unless future datasets support those outcomes. It does not claim that gating interventions causally improve deployed human behavior without a future prospective randomized study.

Recommended Public Release Checklist

1. Confirm that `paper/`, `artifacts/`, and `.private/` are ignored.
2. Run `pytest`.
3. Run `python-mcompileallsrc`.
4. Review `README.md`.
5. Confirm that no manuscript PDFs, source-data CSVs, raw datasets, or submission drafts are staged.
6. Publish only after choosing a repository license.

Summary

The cleaned public repository is an installable, testable, modular research software package. It retains the scientific machinery required to reproduce analyses from public data while keeping unpublished paper materials private. The result is a safer GitHub-ready project: code, tests, commands, and a clear project summary are visible; manuscript and submission artifacts are not.